

预售定金——GMV 前置 + 商家现金流 + 定金不退的法律承诺

两段式支付：定金锁库存 / 尾款转发货 / 失效期没收—gomall v2.3 业务侧 deck

gomall · 业务侧 deck

业务定位：为什么要做预售

三个时间窗口：状态机的业务表达

核心代码：30% 的代码 70% 的业务

业务承诺：定金不退的法律边界

五角色视角：业务全景

业务码与客服话术对照

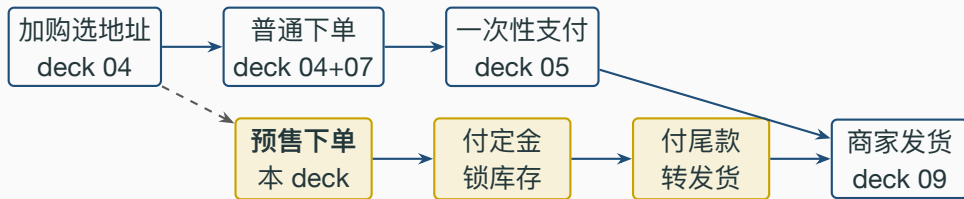
SLO 与压测数据

失败路径与 Saga 补偿

业务边界与路线图

业务定位：为什么要做预售

预售在电商业务全景中的位置



- 预售是**一次性支付链路**的**并行分支**：商品上架时由商家决定是否开预售。
- 上游：预售配置（`product_preorder` 表）由商家后台 / 运营创建。
- 下游：尾款支付后回到 `WaitShip`，与一次性下单合流，履约链路复用 `deck 09`。

`routes/preorder_routes.go` 路由四件套；`service/preorder.go::PayDeposit/PayFinal` 主入口

业务诉求：为什么要拆”定金 + 尾款”

- **C 端**：心理门槛降低。一笔 2000 元商品付 200 元定金 vs 一次性 2000 元，转化率行业平均抬升 **30%-45%**（双 11 / 618 历年数据）。
- **商家**：凭定金销量做备货决策。预售期内 `deposit_paid` 单数 = 准买家数 × 历史转化率（70%-90%），凭这个数下次性原料 / 工厂订单，库存周转 RTM 缩短 20%。
- **平台**：**GMV 前置**。原本 11.11 当天爆发的 GMV 提前到 10.20-11.10 平摊，下单链路负载从 10x 平均 → 3x 平均。
- **合规**：定金不退有法律依据——《民法典》第 587 条”定金罚则”，付定方解约不得请求返还定金。

预售不是”延迟扣款”，是把购买决策拆成两步：买不买（付定金）+ 真要不（付尾款）。

五角色眼中的同一笔预售单

角色	这笔预售单最焦虑的事
C 端用户	” 锁价了吗 / 我能不付尾款吗 / 定金能不能退”
商家	” 预售单量多少 / 我该下多少货 / 退率怎么算”
运营	”GMV 前置比例 / 转化率 (deposit $\square$ final) / 失效率”
客服	三大投诉：定金不退 / 忘付尾款 / 尾款付了又想退
SRE	cron 没收链路有没有延迟 / outbox 是不是发了双份

- 每个角色对应一个或多个业务码 (82001-82004)，客服话术对照表见 §6。
- 本 deck 后面每帧标注它在回答哪个角色的哪个问题。

`pkg/e/code.go 82001-82004`; `pkg/e/msg.go` 客服话术中文

业务边界：本 deck 不做什么

- 不做分期付款：定金 + 尾款是**两次完整扣款**，不是 12 期 / 24 期。分期需对接持牌金融，独立路线图。
- 不做多档定金阶梯：每件商品一个定金 + 一个尾款。”付 200 抵 300 / 付 500 抵 800”这种花式定金是营销侧（路线图）。
- 不做定金返券：用户付完定金不再发**无门槛券 / 满减券**抵尾款。返券会侵蚀财务边界。
- 不做预售退差价：预售期内商品降价后**不补差价**。运营如要补，走人工兜底 + 售后退款。
- 不做预售 SKU 维度：当前一个 `product_id` 一条预售配置。同一商品不同颜色 / 尺寸需共用配置（路线图）。
- 不做预售排队 / 限购：尾款支付不限速；秒杀型预售由 deck 08 营销链路单独承接。

诚实划边界 = 给商家 / 客服 / 法务一个交代。MVP 阶段聚焦”两段扣款 + 状态机推进 + 定金不退”三件事。

三个时间窗口：状态机的业务表达

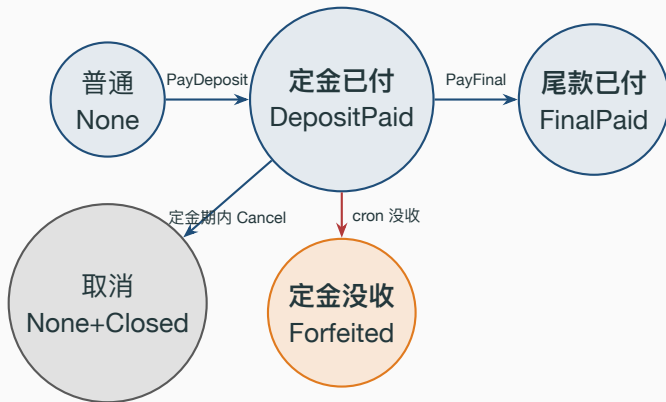
三窗口时间轴：每个窗口能做 / 不能做什么



- **窗口 1 (定金期):** now 落在 DepositStartAt 与 DepositEndAt 之间, 唯一允许 PayDeposit 与 Cancel 全退。
- **窗口 2 (尾款期):** now 落在 DepositEndAt 与 FinalEndAt 之间, 唯一允许 PayFinal; 取消调用返回 **82004 定金不退**。
- **窗口 3 (失效期):** now 大于等于 FinalEndAt, cron ForfeitDepositsForUnpaidFinals 触发, 订单转 Closed, 定金没收。

repository/db/model/preorder.go 字段定义; service/preorder.go::PayDeposit 窗口校验

状态机：order.type + preorder_stage 双维度



- **order.type** 不变 = WaitPay, 直到尾款支付才转 WaitShip; 旧消费者无需感知预售。
- **preorder_stage** 是预售独立维度: 0/1/2/3, 与 type 形成正交。
- Forfeited 是终态, 没有出边; 订单 type 同步推到 Closed。

为什么 type 维持 WaitPay? 兼容性决策

- 业务约束：旧消费者（搜索增量索引 / GMV 报表 / 商家发货筛选）只读 `order.type`。
- 方案 A（侵入式）：新增 `OrderWaitDeposit` / `OrderWaitFinal` 状态码。
- 方案 B（采用）：type 保留 `WaitPay`，新增 `preorder_stage` 字段做“子状态”。
- 选 B 原因：
 - 旧消费者零改动—`WaitPay` 在他们眼里仍是“未付款”，预售订单不会被误推到已发货报表。
 - 状态机表 `orderStateTransitions` 不需要扩 4 个新键，避免 deck 09 17 例转换测试全推倒重来。
 - 风险点：付定金后用户能不能再普通支付？答：单条订单 `paydown` 接口要看 `preorder_stage`，不为 0 时拒绝（路线图加防御）。

consts/order.go 7 态机；service/order_state.go::orderStateTransitions

数据模型：product_preorder 与 order 字段扩展

```
1 type ProductPreorder struct {
2     gorm.Model
3     ProductID uint // 唯一索引：一个商品仅一条预售配置
4     DepositCents int64 // 定金（分）
5     FinalCents int64 // 尾款（分）
6     DepositStartAt time.Time // 定金期开始
7     DepositEndAt time.Time // 定金期结束 / 尾款期开始
8     FinalEndAt time.Time // 尾款期结束，之后 cron 没收
9     ShipAt time.Time // 预计发货时间，仅展示
10 }
11
12 // Order 表扩展：
13 // PreorderStage int 非预售=0 / 已付定金=1 / 已付尾款=2 / 没收=3
14 // DepositPaidAt *time.Time 定金到账时间
15 // FinalPaidAt *time.Time 尾款到账时间
```

- DepositCents + FinalCents 在 order.money 字段累计，与现有 Money 口径一致。
- 三个时间窗口字段是**业务真相**的单一来源，前端 / 客服界面都从 ShowPreorder 读。

核心代码：30% 的代码 70% 的业务

PayDeposit 的 4 步原子操作

```
1 // service/preorder.go::PayDeposit
2 // 1) 时间窗校验: now in [DepositStartAt, DepositEndAt)
3 if now.Before(pp.DepositStartAt) || !now.Before(pp.DepositEndAt) {
4     return nil, newCodedError(e.ErrPreorderNotInDepositWindow)
5 }
6 // 2) Redis 预扣库存 (available -> reserved), 复用 deck 07 两桶 Lua
7 if err := cache.ReserveStock(ctx, req.ProductID, 1); err != nil {
8     return nil, err
9 }
10 // 3) 单一事务内: 建订单 + 扣定金 + 推 stage + 写 outbox
11 err = dao.NewDBClient(ctx).Transaction(func(tx *gorm.DB) error {
12     dao.NewOrderDaoByDB(tx).CreateOrder(order)
13     debitUser(tx, u.Id, req.BossID, req.Key, pp.DepositCents)
14     dao.NewPreorderDaoByDB(tx).MarkDepositPaid(tx, order.ID, now)
15     return dao.NewOutboxDaoByDB(tx).Insert(
16         "order", "PreorderDepositPaid", "preorder.deposit.paid", ...)
17 })
18 // 4) 事务失败 -> 释放 reserved (Saga 补偿)
19 if err != nil { cache.ReleaseReservation(ctx, req.ProductID, 1) }
```

PayFinal: 尾款扣款 + 真扣库存 + 转 WaitShip

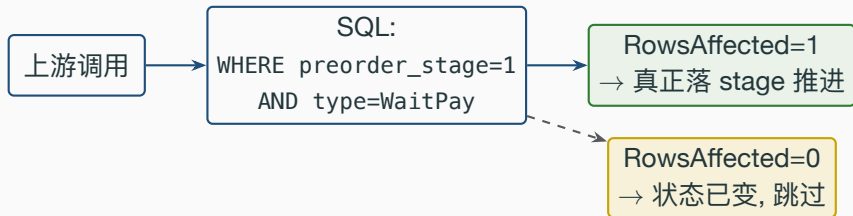
```
1 // service/preorder.go::PayFinal
2 // 1) 业务校验
3 if order.PreorderStage != model.PreorderStageDepositPaid {
4     return nil, newCodedError(e.ErrPreorderDepositNotPaid) // 82003
5 }
6 if now.Before(pp.DepositEndAt) || !now.Before(pp.FinalEndAt) {
7     return nil, newCodedError(e.ErrPreorderNotInFinalWindow) // 82002
8 }
9 // 2) 单一事务内 4 件事: 扣尾款 / 扣 product.Num / 推 stage+type / outbox
10 err = baseDao.DB.Transaction(func(tx *gorm.DB) error {
11     debitUser(tx, u.Id, order.BossID, req.Key, pp.FinalCents)
12     // product.Num -= order.Num (沿用 service/payment.go 口径)
13     productDao.UpdateProduct(...)
14     dao.NewPreorderDaoByDB(tx).MarkFinalPaid(tx, order.ID, now)
15     return outboxDao.Insert("order", "PreorderFinalPaid", ...)
16 })
17 // 3) 事务成功后清 Redis reserved; 失败不回滚 DB (DB 是业务真相)
18 cache.CommitReservation(ctx, order.ProductID, int64(order.Num))
```

cron 没收：单一不可靠靠两层兜底

```
1 // service/preorder.go::ForfeitDepositsForUnpaidFinals
2 // 每小时跑：扫所有 final_end_at 已过但 stage 仍停在 DepositPaid 的订单
3 ids, _ := ppDao.ListUnpaidFinalBefore(nowFn(), 200)
4 for _, id := range ids {
5     s.forfeitOne(ctx, id) // 单笔失败不影响其它
6 }
7
8 // forfeitOne 单笔事务
9 err := baseDao.DB.Transaction(func(tx *gorm.DB) error {
10     // 条件 UPDATE 兜底幂等：stage=1 AND type=WaitPay
11     ok, _ := dao.NewPreorderDaoByDB(tx).ForfeitDeposit(tx, order.ID)
12     if !ok { return nil } // 状态已变（用户卡点付款）→ 跳过
13     return outbox.Insert("order", "PreorderForfeited", ...)
14 })
15 // 事务成功 → 释放 Redis reserved 库存归还
16 cache.ReleaseReservation(ctx, productID, int64(num))
```

*service/preorder.go::ForfeitDepositsForUnpaidFinals; repository/db/dao/preorder.go::ListUnpaidFinalBefore
JOIN*

DAO 层：条件 UPDATE = 状态机 + 幂等



- 一条 SQL 同时校验状态 + 写新状态，避免读后写竞态。
- 上层 ok bool 兜底业务码：ok=false 时一般是”重复触发 / 状态已变”，cron 场景直接 return nil 幂等。
- 与 deck 09 ShipOrder / ConfirmReceive 完全同口径，新人接入零学习成本。

repository/db/dao/preorder.go::MarkDepositPaid/MarkFinalPaid/ForfeitDeposit; deck 09 同口径

Outbox 四事件：业务全景的真实事件流

- 全部走 **Transactional Outbox** (deck 11), 与业务 tx 同提交, 至少一次语义。
- routing_key 命名层级化: preorder.{action}, 方便 RMQ topic 订阅过滤。
- 失效期没收的”定金归属”由钱包 / 财务下游决定 (平台收益 / 商家滞销补贴 / 两边分账), 业务侧只发事件。

service/events/events.go::PreorderXxx 四个 struct; service/outbox/publisher.go 投递

业务承诺：定金不退的法律边界

定金不退：业务承诺 + 法律依据

- 《民法典》第 587 条（定金罚则）：给付定金的一方不履行债务的，无权要求返还定金。
- 平台承诺：用户在首次进入预售页时必须显式确认《预售须知》，包含：
 - 定金支付即视为承诺购买
 - 尾款窗口结束未支付，定金不予退还
 - 商品发货后退款按“已收货”流程执行（deck 09 退款链路）
- 合规要求：监管要求在用户支付定金前必须告知“定金不退”，且字号不得小于其它条款（不能藏在折叠区）。
- 业务码 82004 ErrPreorderForfeitedDeposit 出现在：
 - 预售结束后用户尝试取消 □ 返回 82004
 - 尾款期内用户尝试取消 □ 返回 82004
 - cron 没收后用户查询订单 □ 客服界面显示 82004 + 中文话术

业务承诺写得越清楚，客诉转化为“按规则赔付”的比例越高——这是客服效率指标。

《gomall 预售须知》(用户必读)

1. 本商品为预售商品，定金 **XX 元** + 尾款 **XX 元** = 总价 **XX 元**。
2. 支付定金后视为承诺购买，请在尾款窗口 (**YYYY-MM-DD HH:MM** 至 **YYYY-MM-DD HH:MM**) 内完成尾款支付。
3. 尾款窗口结束未支付，定金不予退还（依据《民法典》第 587 条）。
4. 定金期内可全额退款；尾款支付后退款按平台《售后规则》处理。
5. 预计发货时间为 **YYYY-MM-DD**，以实际发货为准。

- 该弹窗由前端在用户点击“立即付定金”按钮时强制弹出，勾选同意才放支付。
- gomall 后端通过 ShowPreorder 返回的 `deposit_cents / final_cents / now_at / phase` 给前端拼这个文案，避免客户端时钟偏移误判。

`types/preorder.go::PreorderShowResp` 字段；前端实现路线图 `docs/architecture/feature-matrix.md`

五角色视角：业务全景

C 端用户视角：3 个时间窗口看到什么

- **C 端最大焦虑：**忘付尾款 □ 定金没了。解决：尾款期开始前 **24h / 4h / 1h** 三次 **push** 提醒。
- **锁价心理：**用户付 200 元定金的核心动机是”现在锁住 2000 元这个价”，营销文案要强化”双 11 当天补尾款 = 现在的价”。
- **用户故事：**付定金后想退一定金期可全退；尾款期”我先看看其它家” □ 不退；失效期忘付 □ 定金没了。

`service/preorder.go::CancelPreorderInDepositWindow` 取消路径；`events.PreorderCancelled` 通知钱包

商家视角：备货决策的业务化

- 原决策方式：商家凭经验下固定备货量 (e.g. 凭去年同期 + 30%)。库存周转 30 天，超量风险 15%-25%。
- 预售改进：商家在定金期每天看 `deposit_paid` 计数，定金期结束前 1-2 天下次性下单工厂。
- 真实决策：假设定金期共 **1234** 单付定金，历史转化率 (`deposit` → `final`) **75%**：
 - 凭定金销量预估： $1234 \times 0.75 = 925$ 单尾款支付
 - 加 10% 安全冗余：**1020** 件备货
 - 库存周转从 30 天缩到 **7-10 天** (尾款支付集中 + 发货高峰短)
- 商家工作台数据点 (路线图)：`deposit_paid_count / final_paid_count / forfeited_count / conversion_rate`。
- 商家心理账户：定金提前到账 + 尾款 11.11 到账，**现金流改善 7-15 天**，对中小商家是真金白银。

`events.PreorderDepositPaid` 给商家工作台；`events.PreorderForfeited` 给定金归属决策

运营视角：GMV 前置 + 转化漏斗

- **GMV 前置比例**：原本 11.11 当天 100% 爆发 □ 预售期 **30%-40%** GMV 提前落账（定金 + 尾款都算 GMV）。
- **转化漏斗**（双 11 行业平均）：
 - 预售页 PV → 付定金：**2%-5%**（取决于商品热度）
 - 付定金 → 付尾款（成单）：**70%-90%**（爆款上限）
 - 付定金 → 失效（没收）：**5%-15%**（劣品 / 价格回落）
 - 付定金 → 定金期取消：**1%-5%**（用户改主意）
- **运营报表**（路线图）：每个商品的”预售健康度 = 转化率 - 失效率 - 取消率”，作为下次能否开预售的核心指标。
- **异常告警**：单商品失效率 > 30% 触发运营 P2 复盘（是不是商品质量预期落差 / 价格虚高）。

`events.Preorder*` 四事件全量落数据宽表；`docs/architecture/feature-matrix.md` § 运营报表路线图

- 客服界面对每个 82xxx 业务码预置中文话术 + 法律条款链接，平均回复时长 < 2 分钟。
- **硬指标：**客服一通电话内必须给出”定金能 / 不能 / 怎么退”的明确答案，不能拖到工单。

pkg/e/msg.go 82001-82004 中文话术；deck 09 退款流程

SRE 视角: cron 没收的可观测性

- **cron 触发延迟 SLO:** ForfeitDepositsForUnpaidFinals 触发延迟 ≤ 1 小时 (每小时 1 次)。
- **监控指标:**
 - `preorder_forfeit_lag_minutes = now - max(final_end_at)` 中仍 `stage=1` 的订单。 > 60min 触发 P2。
 - `preorder_outbox_publish_latency_p99` `deposit.paid` 事件投递 p99, 与 `deck 11 outbox` 共用。
 - `preorder_forfeit_dead_count` `outbox` 走 `dead` 的 `forfeited` 事件数, > 0 触发值班。
- **真实故障假设:**
 - `cron` 进程崩 / 进程未起 □ 失效订单滞留 `stage=1` (用户能“补付”窗口实际拖长) □ 监控滞后 60min 报警。
 - `outbox` 事件丢 □ 商家不知道哪些定金被没收 □ 工作台 `deposit_paid` 计数虚高 □ 错下货。
 - 数据库时钟漂移 □ `final_end_at` 判定不准 □ **服务端时钟必须 NTP 强制同步。**

`service/preorder.go::nowFn` 时钟函数指针; `service/preorder.go::forfeitOne` 单笔事务

业务码与客服话术对照

- **业务码命名规则：**8 段位 = 业务侧（不是技术错误），第 2-3 位是功能模块（20= 预售），后 2 位是子场景。
- 与其它业务码不冲突：60xxx 幂等 / 70xxx 限流熔断 / 80xxx 满减 / 81xxx 拼团 / 82xxx 预售。
- 业务码透出方式：`service.CodeOf(err)` 提取，
`api/v1/preorder.go::respondPreorderErr` 标准化返回。

pkg/e/code.go:75-80 82xxx 段；api/v1/preorder.go::respondPreorderErr 业务码透出

业务码客服话术（中文版）

```
1 // pkg/e/msg.go
2 ErrPreorderNotInDepositWindow: "当前不在预售定金期，无法支付定金",
3 ErrPreorderNotInFinalWindow: "当前不在尾款支付窗口，无法支付尾款",
4 ErrPreorderDepositNotPaid: "尚未支付定金，无法支付尾款",
5 ErrPreorderForfeitedDeposit: "预售已结束，根据预售须知定金不予退还",
```

- 客服一线话术与业务码一一对应，避免一线自由发挥说错话。
- 诚实：82004 必须明确告知“不予退还”四个字—模糊话术（“请等待处理”）反而引发更多次电话。
- 客服后台日均统计每个业务码出现次数 □ 反推产品 / 运营优化方向：
 - 82001 多 □ 用户分不清窗口期，加日历提醒
 - 82004 多 □ 失效率高，复盘商品价格 / 营销定位

`pkg/e/msg.go` 完整 4 条；客服系统对接路线图 `docs/architecture/feature-matrix.md`

SLO 与压测数据

SLO: 每个接口的业务承诺

- **定金接口 < 200ms**: 链路 = Redis Lua reserve (5ms) + DB Tx 写订单 + AES 扣款 (30-50ms) + outbox 写 (5ms)。
- **尾款接口 < 500ms**: 多一次 product.Num 更新 + 状态机推进 SQL, 留余量 200ms 防 GC / 网络抖动。
- **cron 1h**: 尾款期一般是数天 - 周, 没收延迟 1h 业务可接受; 缩短到分钟级需独立 cron 进程, 路线图阶段。

stressTest/REPORT.md /orders/create 基线 50K RPS / p99 2.33ms 复用为参考

压测参考：复用一次性下单的基线数据

- **基线**：/ping **64,254 RPS / p95 3.51ms**（裸 gin 链路）—stressTest/REPORT.md §1
- **/orders/create**（幂等 + 库存 + outbox）：**50,319 RPS / p95 2.33ms**，755,033 次重试 1 笔订单
- **/coupon/claim**（Redis Lua）：**51,362 RPS / p95 3.52ms**，500 抢 100 张零超发
- 预售/preorder/:productID/deposit 链路结构等同 orders/create + 1 次 AES 扣款：
 - 预估 RPS：**40K-45K**（多一次 user UpdateMoney）
 - 预估 p95：**4-5ms**
 - 预估 p99：**< 50ms**（远低于 SLO 200ms）
- **cron 没收链路**：单批 200 条 / 每小时 1 次。失效订单 200 条以内的商家 cron 单次 < **500ms**。
- **Redis reserved 桶占用**：每笔定金占 1 件，全平台 100 万预售单 = 100 万 Redis key (< 50MB)，可忽略。

stressTest/REPORT.md §1 基线；service/payment.go::PayDown AES 链路实测复用

真实数字：双 11 历年预售规模参考

- 行业历史数据（业务侧公开报告）：
 - 2023 双 11 预售期 **10.20-11.10** GMV 占大促总 GMV **38%**
 - 预售单平均定金占比 **8%-15%**（200 元商品定金 20-30 元）
 - 单日定金接口 QPS 峰值 **50-80K**（开预售 0 点和接近定金期末两个高峰）
 - 尾款支付 0 点 QPS 峰值 **200-400K**（远高于定金）
- 对 gomall 的启示：
 - 定金链路当前 SLO 余量充足（45K RPS 容量 vs 80K 峰值需求）□ 异步削峰路径（deck 10）需要扩到 preorder
 - 尾款链路峰值远超基线 □ 需要类似 `/orders/enqueue` 的异步路径（路线图）
 - Redis reserved 桶在双 11 前 1-2 天会涨到日常 50-100 倍，要给 Redis 扩容预留容量

`stressTest/REPORT.md`；`deck 10` 异步削峰章节；`service/order_async.go` 异步路径复用方向

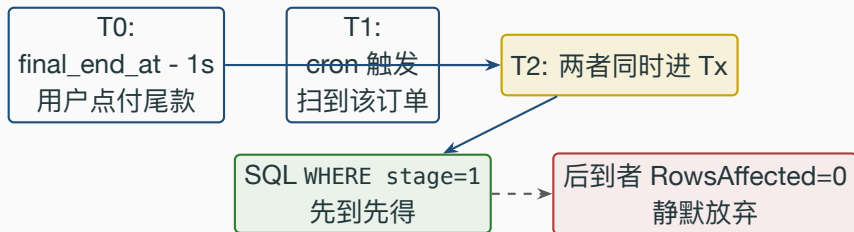
失败路径与 Saga 补偿

Saga 视角：4 个故障路径的兜底

- 补偿原则：DB 是业务真相，cache 是性能侧。Cache 失败不回滚 DB（与 deck 11 ProductUpdate 双删策略同口径）。
- 至少一次语义：outbox 投递可能重发，下游消费者必须幂等（路线图：钱包消费 preorder.* 时按 order_id 去重）。
- **cron 单笔幂等**：ForfeitDeposit SQL WHERE stage=1 AND type=WaitPay 兜底，重复触发 RowsAffected=0 无副作用。

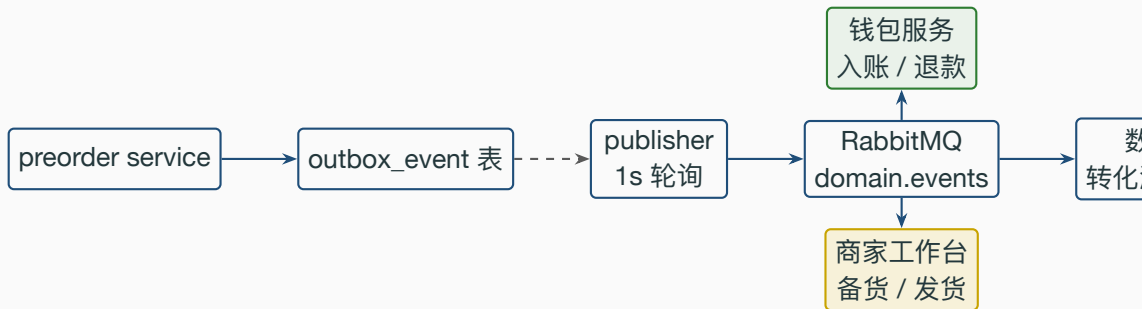
service/preorder.go::PayDeposit 失败路径释放；*service/order_cancel.go* 同口径 *release*

真实事故假设：用户卡点付款 vs cron 没收的竞态



- 竞态场景：用户在 `final_end_at - 1s` 点付尾款，正好 cron 也在同一秒触发。
- 解决：两者都走 `WHERE preorder_stage=1 AND type=WaitPay` 条件 UPDATE，数据库行锁兜底。
 - 用户付尾款先 commit □ stage 2 □ cron `ForfeitDeposit RowsAffected=0` □ `forfeitOne return nil`
 - cron 先 commit □ stage 3 □ 用户 `MarkFinalPaid RowsAffected=0` □ 返回” 订单状态已变更”
- 业务结论：先到先得，绝不双扣。用户兜底场景应在前端引导” 提前 5min 内不再开放支付” 以避免边缘 case。

业务全景：4 个 outbox 事件接到哪



- 钱包消费 `preorder.deposit.paid`: 用户钱包扣 X 元、商家钱包加 X 元（与 AES 扣款冗余记账）。
- 商家工作台消费 `preorder.final.paid`: 进入待发货 tab; forfeited 进入”没收订单”统计 tab。
- 数据宽表消费所有 4 个事件: 算转化漏斗 / 失效率 / 客单价。
- 至少一次: 消费者按 `order id + event type` 做幂等键、重复消息直接 ON CONFLICT

业务边界与路线图

路线图：本期之后做什么

- 当前版本聚焦” 两段扣款 + 状态机推进 + 定金不退” 三件事，其它是路线图阶段。

docs/architecture/feature-matrix.md 路线图分级表； *deck 08* 满减引擎

诚实自爆：当前实现的取舍点

- **1 单 1 件硬约束**：当前 PayDeposit 强制 Num=1，多件下单需走普通链路。理由：预售场景下用户决策粒度天然就是“单件商品”，多件并发付款会引入 SKU + 库存配对复杂度。
- **cron 表达式**：本 deck 未引入 preorder cron（路线图阶段加 initialize/cron.go 的 ForfeitDepositsForUnpaidFinals 调用）。当前未自动触发，需手动调用 service 入口测试。
- **没收资金归属未定**：PayDeposit 时定金已划给商家，Forfeited 事件只标状态、不动钱包。”定金应归平台还是商家还是分账”留给 wallet 服务消费 preorder.forfeited 时决定。
- **AddressID 可选**：定金期内允许不填地址，尾款支付前补地址。当前未强制校验补全，路线图加防御。
- **时钟函数指针**：nowFn 用 var 而非 context.Context 携带，方便测试替换但生产侧没必要切。改成 ctx 透传是更优雅的路线图改造。

service/preorder.go::PayDeposit Num=1; service/preorder.go::nowFn 测试钩子

面试 Q&A (上)

Q1: 为什么用 `preorder_stage` 字段而不是扩 `order.type` 状态?

A: 旧消费者只读 `type`, 扩 `type` 要改一票下游; `stage` 做正交子状态, 对旧消费者透明
— 见 `deck 09 17` 例状态转换测试零改动。 *repository/db/model/order.go:24*

Q2: 定金期内取消, 怎么保证“定金真的退回”?

A: `CancelPreorderInDepositWindow` 在事务内调 `refundUser` (与 `debitUser` 反向 AES), 同事务写 `outbox preorder.cancelled` 给钱包二次记账, 双保险。

service/preorder.go::CancelPreorderInDepositWindow

Q3: 用户在 `final_end_at - 1s` 付尾款, `cron` 同时触发, 谁赢?

A: 先 `commit` 者赢 — 双方走条件 `UPDATE WHERE stage=1 AND type=WaitPay`, 数据库行锁兜底。后到者 `RowsAffected=0`, 业务上自然降级 (用户付款 □ 用户报错 / `cron` □ 静默跳过)。 *repository/db/dao/preorder.go::MarkFinalPaid/ForfeitDeposit*

Q4: 定金 + 尾款是两次扣款 vs 冻结 + 扣款, 为什么选两次?

A: 冻结需要支付通道 (支付宝 / 微信) 支持“预授权”接口, `gomall` 当前不真接第三方支付, AES 模拟扣款, 两次扣款链路更简单, 更与现有 `pay` 一致, 生产按逻辑授权后

面试 Q&A (下)

Q5: cron 1 小时一次是不是太慢?

A: 尾款期一般是 1-7 天, 1h 没收延迟业务可接受。缩到 1min 需要独立 cron 进程 + 防扫表压力, 对业务**无明显收益**。SLO 写 1h 是诚实边界。*deck 09 cron* 双保险关单同口径

Q6: 商家凭定金销量备货, 转化率 75% 是怎么算出来的?

A: 是**历史经验值** (双 11 行业平均 70%-90%)。gomall 当前没有真实历史数据, 路线图阶段先按 75% 给商家做”建议下单量”, 3-5 期之后用商家自己的历史 `conversion_rate` 替换。*events.PreorderDepositPaid* 给商家工作台

Q7: 定金不退是不是侵犯消费者权益?

A: 法律明确支持 (《民法典》第 587 条定金罚则)。**合规要点:** 必须在用户支付定金前明确告知, 字号不得小于其它条款。gomall 通过 `ShowPreorder` 返回的字段强制前端拼《预售须知》弹窗。*types/preorder.go::PreorderShowResp* 字段给前端

Q8: 定金已经划给商家了, cron 没收时为什么不再扣商家?

A: 业务上没收的”归属”是**独立决策**。当前版本只标 `preorder_stage=Forfeited` + 发事件, 钱包/财务下游决定”归商家/归平台/公账”, 这是业务条线上, 故意不写死

代码位置一览

预售配置表 *repository/db/model/preorder.go:18-26*

Order 字段扩展 (PreorderStage / DepositPaidAt / FinalPaidAt) *repository/db/model/order.go*

DAO (GetPreorder / Mark* / Forfeit / List*) *repository/db/dao/preorder.go*

PayDeposit 主入口 (窗口 + Reserve + Tx + outbox) *service/preorder.go::PayDeposit*

PayFinal 主入口 (扣尾款 + 真扣库存 + 状态机) *service/preorder.go::PayFinal*

CancelInDepositWindow (全退路径) *service/preorder.go::CancelPreorderInDepositWindow*

ForfeitDepositsForUnpaidFinals (cron 入口) *service/preorder.go::ForfeitDepositsForUnpaidFinals*

ShowPreorder (公开查询) *service/preorder.go::ShowPreorder*

HTTP handler 四件套 *api/v1/preorder.go*

路由注册 *routes/preorder_routes.go*

业务码 82001-82004 *pkg/e/code.go: 78-83; pkg/e/msg.go*

outbox 事件 struct *service/events/events.go::PreorderXxx* 四个 struct

单测覆盖 8 例 *service/preorder_test.go*

配套：deck 04 下单 / deck 05 支付 / deck 07 库存 / deck 09 订单生命周期 /
deck 11 outbox 与一致性。